

[5. Špecifické situácie](#)

[5.1 Kompozícia](#)

[Príklad - Nákupný košík](#)

[5.2 Asociácia 1:1](#)

[Príklad - Povinné zmluvné poistenie auta](#)

[5.3 Mapovanie dvoch tried do jednej tabuľky](#)

[Príklad - Povinné zmluvné poistenie auta, verzia 2.](#)

[5.4 Mapovanie jednej triedy do viacerých tabuliek](#)

[Príklad - Osoba a jej kontaktné údaje](#)

[5.5 Zložené kľúče](#)

[Príklad - Zdravotné záznamy](#)

[Cvičenia](#)

5. Špecifické situácie

Prednášky sa budeme venovať mapovaniu v niektorých neštandardných situáciách:

- kompozícia
- 1:1 asociácia
- mapovanie dvoch/viacerých tried do jednej tabuľky
- mapovanie jednej triedy do viacerých tabuliek
- zložené kľúče

5.1 Kompozícia

Kompozícia je špeciálny prípad `OneToMany` asociácie, v ktorej **asociované objekty - komponenty môžu existovať len ako súčasť celku - kontajnera**.

Príklad - Nákupný košík

[Pozri projekt p51composit](#)

Nákupný košík obsahuje - vlastní viacero položiek s vybraným tovarom. Položky môžu existovať len ako súčasť košíka, ide teda o vzťah kompozície (ako ho definuje UML)



Asociácia medzi kontajnerom a komponentami sa zvyčajne realizuje ako jednosmerná: kontajner pozná spravované komponenty, ale komponenty nemusia poznať kontajner, v ktorom sú. Kardinalita je spravidla 1:N, nemusí byť v diagrame vyznačená.

Entitné triedy

```
@Entity
public class Polozka implements Serializable {
    @Id
    private Long id;

    private String tovar;
    private int mnozstvo;
}
```

```
@Entity
public class Kosik implements Serializable {
```

```

@Id
private Long id;

@OneToMany(cascade=CascadeType.REMOVE, orphanRemoval=true)
@JoinColumn(name="KOSIK_FK")
private List<Polozka> polozky;

```

Parameter `cascade=CascadeType.REMOVE` zabezpečí, že pri odstránení košíka z databázy budú s ním automaticky z databázy odstránené aj všetky položky.

Parameter `orphanRemoval=true` zabezpečí, že ak sa položka odstráni z košíka (zo zoznamu položky) bude nielen vynulovaná FK hodnota v tabuľke KOSIK, ale odstránený aj záznam z tabuľky POLOZKA. (samotné `CascadeType.REMOVE` nestačí)

Aplikácia obsahuje tri funkcie

`create` - vytvorí košík s niekoľkými položkami. (Jej spustením môžeme vygenerovať aj tabuľky.)

`removeKosik` - odstráni košík aj s položkami

`removePolozka` - odstráni položku z košíka

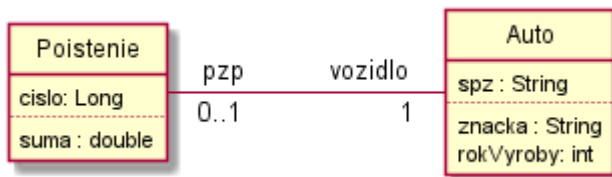
Relačný diagram štruktúra databázy je rovnaká ako pri obvyčajnej `OneToMany` asociácii.

5.2 Asociácia 1:1

Príklad - Povinné zmluvné poistenie auta

Pozri projekt [p52bi11](#)

Typickou ukážkou 1:1 asociácie medzi dvoma entitnými triedami je vzťah medzi vozidlom a jeho povinným poistením:



Entitné triedy

Pre 1:1 asociácie má JPA anotáciu `@OneToOne`. Takto anotovaný atribút bez ďalších JPA anotácií a parametrov je namapovaný na samostatný FK stĺpec, teda štruktúra databázy je rovnaká ako pri `@ManyToOne`.

```

@Entity
public class Poistenie implements Serializable {
    @Id
    private Long cislo;
    private double suma;
    @OneToOne
    @JoinColumn(nullable=false) // vozidlo 1
    private Auto vozidlo;
}

```

```

@Entity
public class Auto implements Serializable {
    @Id
    private String spz;
    private String znacka;
    private int rokVyroby;
    @OneToOne(mappedBy="vozidlo")
    private Poistenie pzp;
}

```

Poznámky:

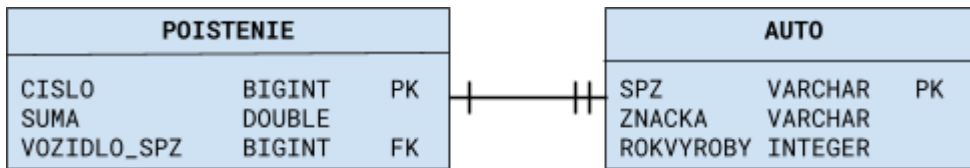
- Parameter `nullable=false` v anotácii referencie na vozidlo zabezpečuje, že poistenie musí mať zadané vozidlo (v databáze), ako vyžaduje v UML kardinalita 1 na strane vozidla.
- Vlastník (owner) asociácie môže byť ktorákoľvek strana.

- Asociácia by mohla byť tiež jednosmerná.

Relačný diagram

Skutočnosť, že ide o vzťah `OneToOne` vyjadruje v diagrame symbol `|` na strane poistenia (namiesto kuracích papáčiek).

Štruktúra databázy je však podobná ako pri `ManyToOne` asociácii, len FK stĺpec by mal mať definovaný `unique constraint`, (overiť si to môžete vo vlastnostiach indexu na stĺpci).



5.3 Mapovanie dvoch tried do jednej tabuľky

Príklad - Povinné zmluvné poistenie auta, verzia 2.

[Pozri projekt p53embed](#)

Povinné zmluvné poistenie môžeme modelovať kompozíciou s kardinalitou 1:1.

Poistenie bude kontajner a objekt obsahujúci informácie o poistenom aute je jeho komponenta.



Entitné triedy

Keďže informácie o poistenom aute sú súčasťou poistenia, môžeme ich atribúty v databáze ukladať priamo do tabuľky `POISTENIE`. V tomto prípade je len trieda `Poistenie` entitná, trieda `Auto` je vnorená.

- Pre anotáciu referencie na asociovaný objekt vnorenej triedy sa použije `@Embedded`
- Pre anotáciu vnorenej triedy sa použije `@Embeddable` (pričom nemá PK atribút)

```
@Entity
public class Poistenie implements Serializable {
    @Id
    private Long cislo;
    private double suma;
    // @OneToOne
    @Embedded
    private Auto vozidlo;
```

```
@Embeddable
public class Auto implements Serializable {
    private String spz;
    private String znacka;
    private int rokVyroby;
```

Poznámka: `Auto` už, nie je entitná trieda. Vložiť do databázy ho možno len ako súčasť poistenia, teda nie je pre neho ani potrebné volať `persist`.

Relačný diagram má len jednu tabuľku

POISTENIE		
CISLO	BIGINT	PK
SUMA	DOUBLE	
SPZ	VARCHAR	
ZNACKA	VARCHAR	
ROKVYROBY	INTEGER	

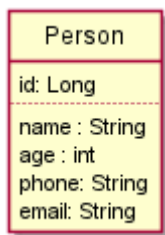
5.4 Mapovanie jednej triedy do viacerých tabuliek

Opačná situácia je, keď chceme atribúty entitnej triedy rozdeliť do viacerých tabuliek.

Príklad - Osoba a jej kontaktné údaje

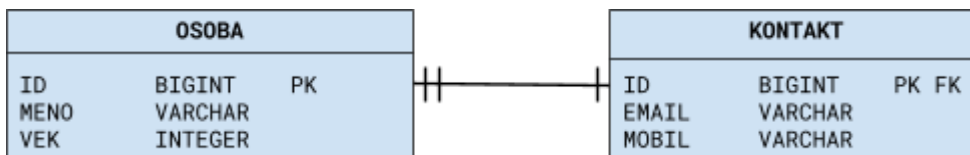
Pozri projekt [p54secondary](#)

V aplikácii chceme mať všetky informácie o osobe sústredené v jedinej entitnej triede Person.



Relačný diagram

V databáze však budú rozdelené do dvoch tabuliek OSOBA a KONTAKT, v ktorých majú záznamy o osobe a jej kontaktoch rovnaké kľúče.



Všimnite si, že primárny kľúč podriadenej tabuľky je zároveň cudzím kľúčom do primárnej tabuľky entity. Tabuľky sú teda vo vzťahu 1:1.

Entitné triedy

Entitnú triedu mapujeme na dve tabuľky, pričom podriadená tabuľka bude sekundárna.

```

@Entity
@Table(name = "OSOBA")
@SecondaryTable(name="KONTAKT")
public class Person implements Serializable {
    @Id
    private Long id;
    @Column(name="MENO")
    private String name;
    @Column(name="VEK")
    private int age;
    @Column(table= "KONTAKT")
    private String email;
    @Column(table= "KONTAKT", name="MOBIL")
    private String phone;
  }
  
```

Názov sekundárnej tabuľky zadáme pomocou anotácie entitnej triedy `@SecondaryTable`. Meno tabuľky aj stĺpca, na ktorý je konkrétny dátový člen namapovaný, určíme pomocou parametrov anotácie `@Column`.

5.5 Zložené kľúče

Tabuľky so zloženými kľúčmi sa uplatňujú a používajú v rôznych situáciách:

- pri kompozícii keď primárny kľúč komponenty je zložený z primárneho kľúča kontajnera a identifikátora komponenty v rámci kontajnera (unikátneho len v rámci kontajnera)

- pri entitách, ktoré treba mať uložené vo viacerých verziách
- pri historizovaných entitách, ktorý si okrem aktuálneho stavu uchovávajú aj predchádzajúce.

Zložený kľúč má v takom prípade dve zložky: identifikátor entity a číslo verzie resp. údaj o čase.

Príklad - Zdravotné záznamy

Pozri projekt [p55compkey](#)

Relačný diagram

Každodenné záznamy o teplote, tlaku ... pacienta počas jeho liečenia je vhodné ukladať v tabuľke so zloženým kľúčom.

ZAZNAM		
PACIENT	VARCHAR	PK
DATUM	DATE	PK
TEPLOTA	DOUBLE	
PULZ	DOUBLE	

Entitné triedy

Ak má tabuľka zložený kľúč, je potrebné okrem entitnej triedy vytvoriť aj ďalšiu triedu, ktorá obsahuje atribúty pre kľúčové stĺpce tabuľky. Trieda reprezentujúca kľúč je `@Embeddable`

```
@Embeddable
public class ZaznamPK implements Serializable {
    private pacient String;
    @Temporal(TemporalType.DATE)
    private Date datum;
}
```

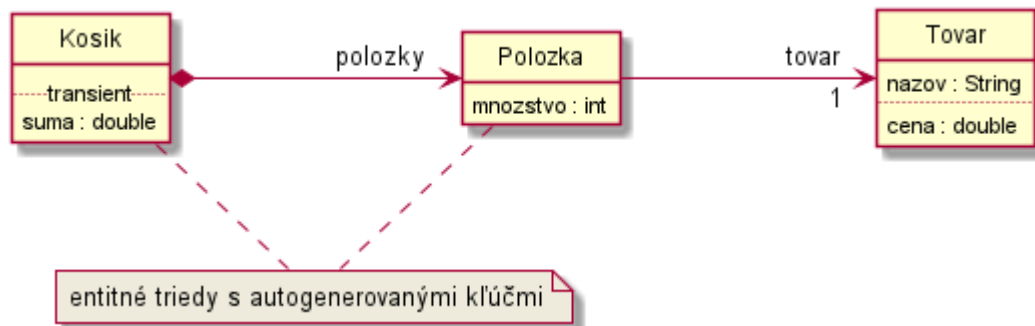
V entitnej triede sa použije na označenie kľúča namiesto `@Id` anotácia `@EmbeddedId`

```
@Entity
public class Zaznam implements Serializable {
    @EmbeddedId
    private ZaznamPK pk;
    private double teplota;
    private double pulz;
}
```

Cvičenia

Cvičenie 1. Prerobte projekt [p51composit](#) tak aby bola asociácia obojsmerná.

Cvičenie 2. Reimplementujte triedy nákupného košíka podľa UML diagramu:



Tovar bude samostatná entitná trieda

Triedy Kosik a Polozka majú tabuľky s autogenerovanými kľúčmi (nie sú vyznačené v diagrame). Košík má atribút suma, ktorý udáva aktuálnu celkovú cenu tovarov v košíku. Tento atribút sa nezapíše do databázy (je transient)

Implementujte dve funkcie:

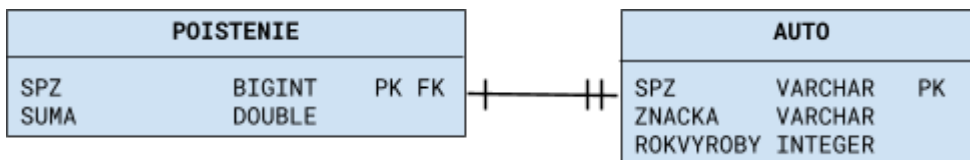
- `pridajTovar(Kosik k, Tovar t)`
Pridá tovar do košíka: Ak tovar v košíku ešte nie je, pridá novú položku. Ak už v košíku tovar bol, zvýši jeho množstvo.
- `odoberTovar(Kosik k, Tovar t)`
Odoberie tovar z košíka, t.j zníži množstvo tovaru o 1. Ak klesne množstvo na 0, odstráni položku. Ak už košík nebude obsahovať žiadne položky, odstráni ho z databázy.

Cvičenie 3.

- A. Skúste, či je do databázy vytvorenej v projekte [p52bi11](#), je možné vložiť (manuálne alebo aplikáciou) viac poistení pre to isté auto.
Ak je to možné, zrejme JPA nevygeneroval v databáze unique constraint na FK stĺpec, aj napriek anotácii `OneToOne`.
Pridajte referencii na `vozidlo` ešte anotáciu `JoinColumn(unique=true)` a pregenerujte databázu. Teraz by už malo vytvoriť unique-index pre FK, ktorý nedovolí vložiť viac poistení pre jedno auto.
- B. Upravte projekt tak, aby bola asociácia jednosmerná: Poistenie → Auto (stačí odstrániť v aute referenciu `pzp`).
V aplikácii vytvorte dve zmluvy pre to isté auto a zistite, či sa ich podarí vložiť do databázy.

Cvičenie 4. (nepovinné)

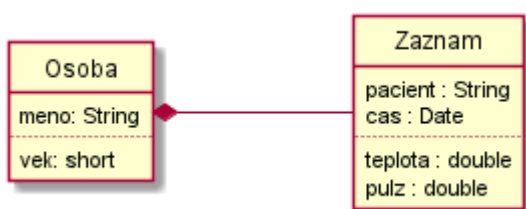
1:1 asociáciu medzi poistením a autom by bolo možné namapovať na databázu tak, že SPZ by slúžila ako primárny kľúč, nielen v tabuľke AUTO ale aj v tabuľke POISTENIE.



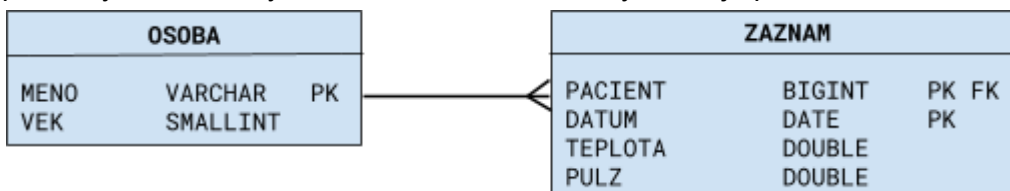
Pozri http://www.java2s.com/Tutorials/Java/JPA/0730_JPA_OneToOne_Primary_Key_Join.htm

Cvičenie 5. V projekte [p53embed](#) implementujte a funkciu `List<Poistenie> findByYear(int year)`, ktorá pomocou JPQL-dotazu vyhladá všetky poistné zmluvy pre autá so zadaným rokom výroby.

Cvičenie 6. Do projektu [p55compkey](#) doplňte entitnú triedu `Osoba` vytvoriť kompozíciu podľa UML diagramu:



Pomocou JPA anotácií namapujte entitné triedy na databázu, ako je znázornená na diagrame. Tabuľka ZAZNAM má zložený kľúč a jeho prvá zložka je zároveň cudzí kľúč odkazujúci na primárny kľúč tabuľky OSOBA, teda meno osoby, ktorá je pacientom.



Návod. Ak nebudete vedieť, ako správne dať anotácie, aby sa vygenerovala databáza podľa diagramu, skúste opačný postup: Pridajte do databázy vytvorenej projektom `p55compkey` tabuľku OSOBA a tabuľke ZAZNAM pridajte foreign key `PACIENT → OSOBA.MENO`. V novom projekte vygeneruje wizardom z databázy entitné triedy.
Pozri tiež [anotáciu @MapsId](#).

Cvičenie 7. Práca s historizovanou tabuľkou

Organizácia eviduje v databáze údaje o svojich zamestnancoch ako meno, pracovné zadanie, plat... Prirodz

ene viaceré z týchto údajov sa môžu časom meniť. Aby bolo možné zistiť nielen aktuálny stav (napr. plat) ale aj stav kedykoľvek v minulosti, sú údaje uložené v tzv. historizovanej tabuľke, v ktorej kľúč je zložený z dvoch stĺpcov:

- ID LONG - identifikátor entity - zamestnanca
- OD TIMESTAMP - čas zmeny, t.j odkedy platia nové údaje:

Ďalej tabuľka obsahuje historizované údaje o zamestnancovi, viď diagram:

ZAMESTNANEC		
ID	LONG	PK
OD	TIMESTAMP	PK
MENO	VARCHAR	
PLAT	DOUBLE	
ZARADENIE	VARCHAR	

Implementujte entitnú triedu pre historizovanú tabuľku ZAMESTNANEC. Implementujte a otestujte CRUD funkcie pre prácu s informáciami o zamestnancoch

- `vyhladatZamestnanca(long id)` vráti aktuálne údaje o zamestnancovi
- `vyhladatZamestnanca(long id, Date d)` vráti údaje o zamestnancovi platné, k danému dátumu
- `prijatZamestnanca(long id, String meno)` vytvorí nového zamestnanca
- `aktualizovatZamestnanca(Zamestnanec z)` aktualizuje údaje o zamestnancovi
- `prepustitZamestnanca(Zamestnanec z)` prepustí zamestnanca

Dôležité: V historizovanej tabuľke sa nikdy záznamy nevymazávajú ani nemodifikujú, len vkladajú. To značí, že:

- Funkcia `aktualizovatZamestnanca` musí namiesto UPDATE volať INSERT, ktorý pridá nový záznam s aktualizovanými údajmi do tabuľky ZAMESTNANEC. Kľúčový stĺpec OD bude obsahovať timestamp okamihu, keď nastáva zmena (použite `new Date()`)
- Funkciu `prepustitZamestnanca` môžeme implementovať ako špeciálny prípad `aktualizovatZamestnanca`, pričom hodnoty zo stĺpcov MENO, PLAT a ZARADENIE vymažeme.